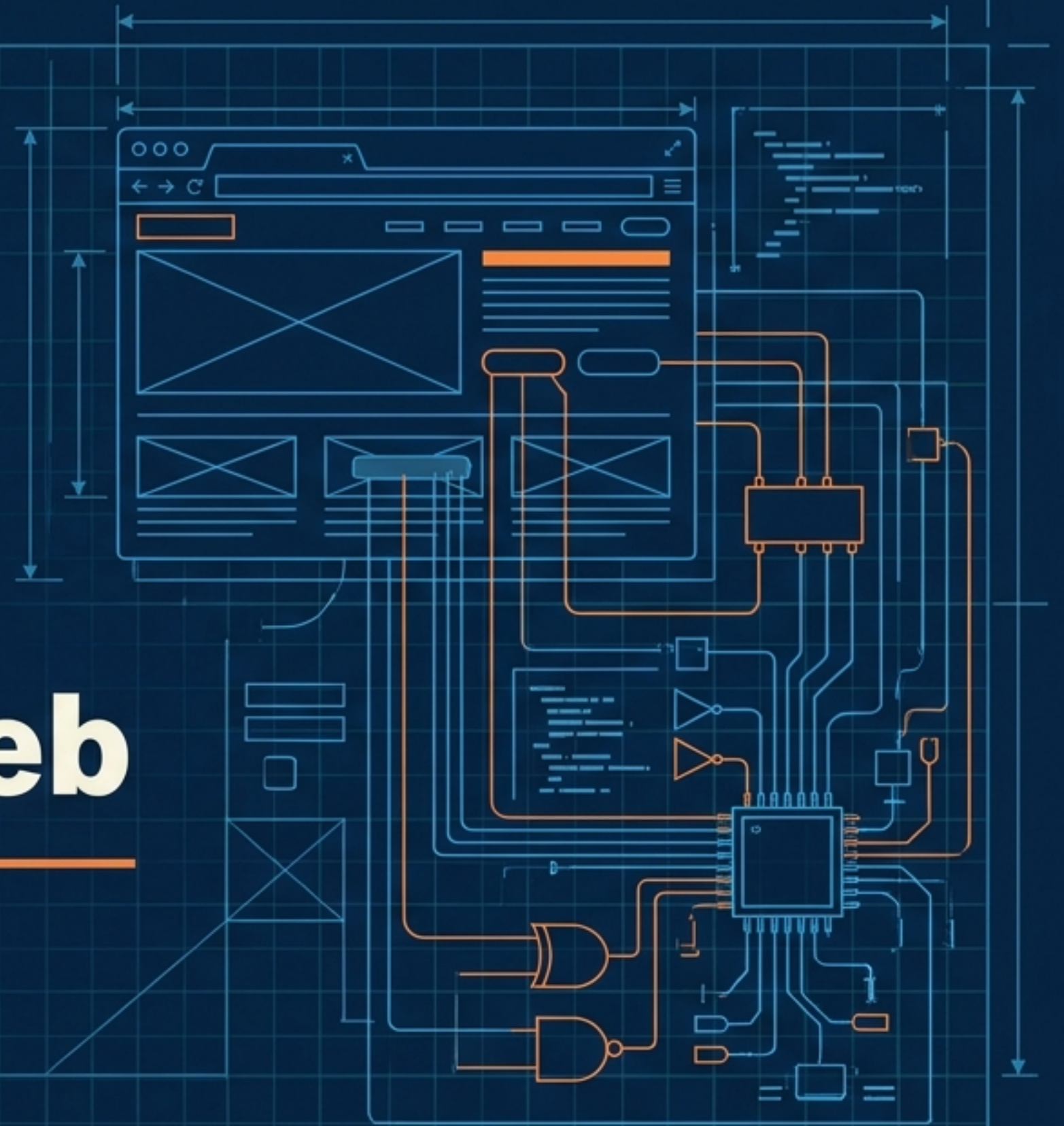
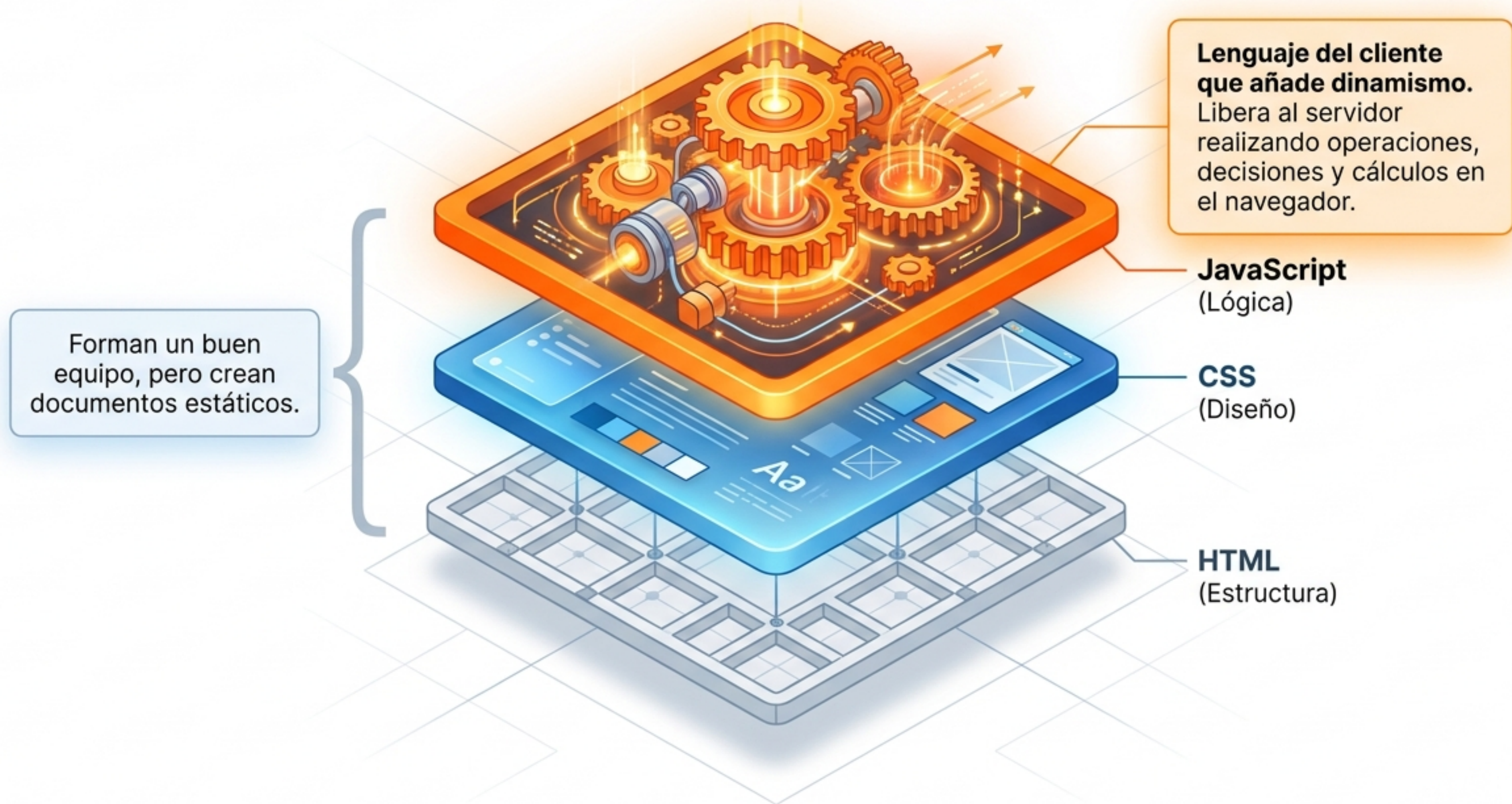


Introducción a JavaScript (I) El Motor de la Interactividad Web

Fuente: Lenguaje de marcas y sistemas de gestión de información. Tema 18.



El Eslabón Perdido: Del Documento a la Aplicación



Matriz de Integración: ¿Dónde vive el código?

1. En el Elemento (Etiqueta)

```
<p onclick='...'></p>
```

Acciones aisladas e inmediatas.

2. En el Documento (Head/Body)

```
<script type="text/javascript">  
  console.log("Lógica de página");  
</script>
```

Lógica específica para una sola página. Se carga en el head o al final del body.

3. Fichero Externo (Recomendado)

```
<script type="text/javascript"  
  src="/js/codigo.js"></script>
```

Mejor práctica. Código reutilizable y estructurado.

Dos Interfaces: Desarrollador vs. Usuario

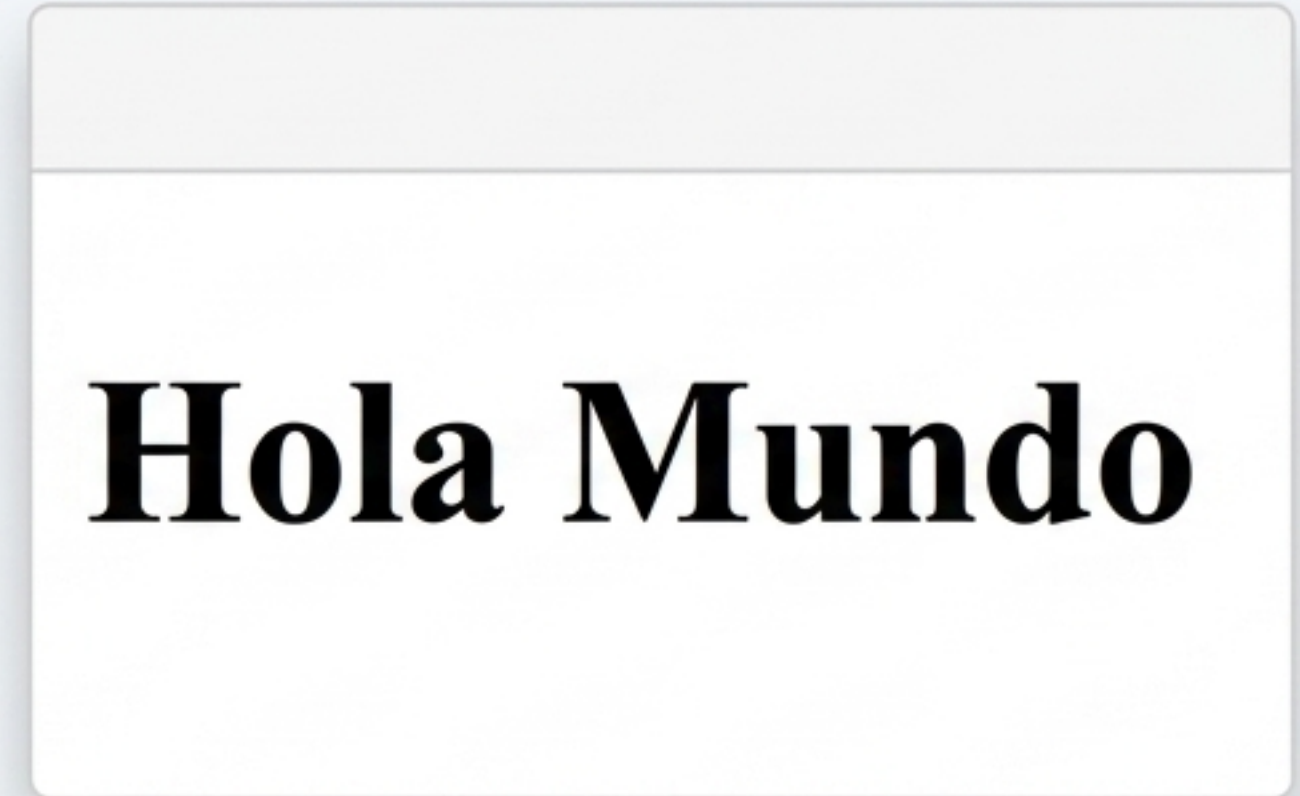
Modo Desarrollador



Herramienta: `console.log()`

Depuración (debugging), evaluar variables y comprobar flujo.
Tip: `console.clear()` para limpiar.

Modo Usuario

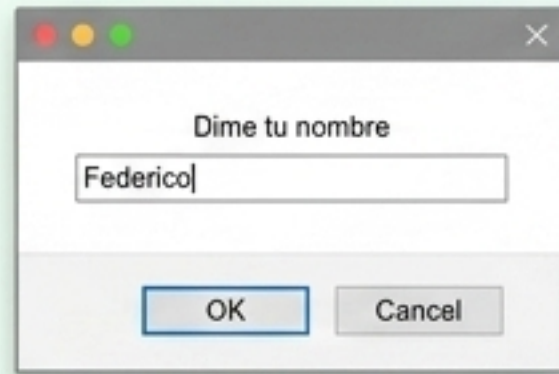


Herramienta: `document.write()`

Escribir etiquetas HTML e información directamente en la web visible.

Ciclo de Interacción Básica (I/O)

1. Entrada



```
prompt('Mensaje', 'Defecto')
```

Pide datos al usuario.
Devuelve el valor o null.

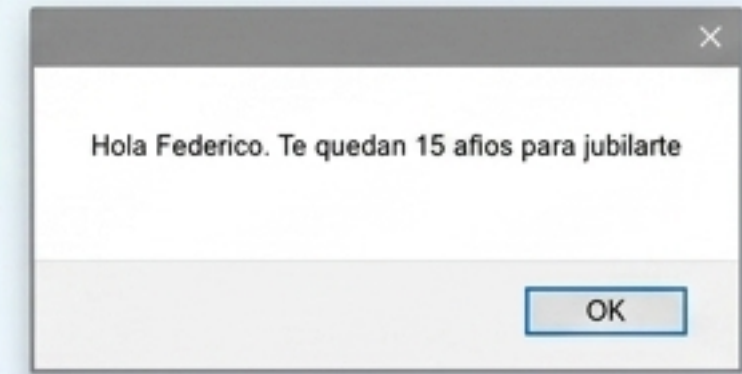
2. Memoria



`var`

Asignación a variables.
El motor guarda el dato.

3. Salida



```
alert('Información')
```

Muestra una caja emergente
de confirmación.

Anatomía de una Variable (Memoria Dinámica)

Declaración.
Recomendada, aunque JS la crea automáticamente si se omite.

**var resultado =
numero1 + numero2;**

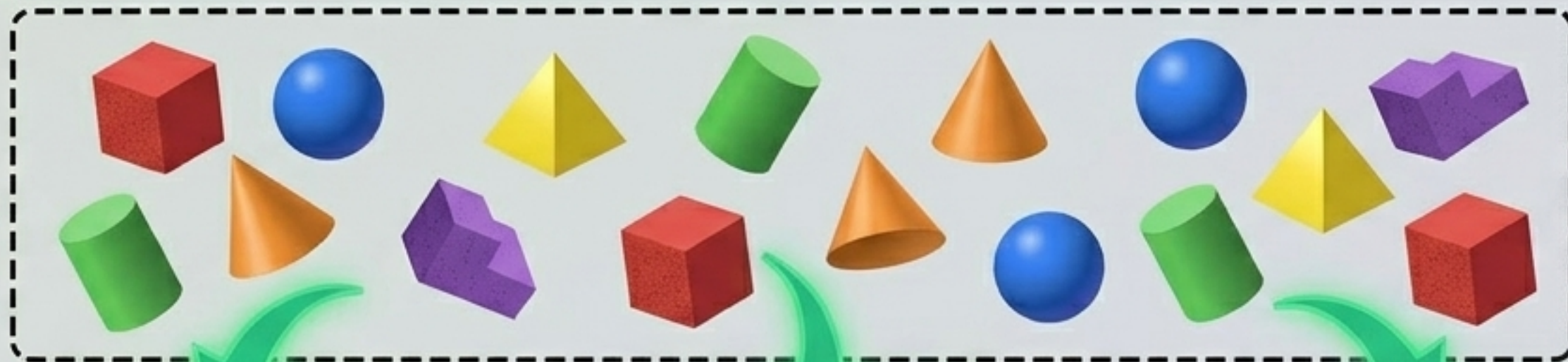
Inicialización.
Asignación de
valor.

Reglas de Nomenclatura.
Letras, números, \$ y _.
Sensible a
mayúsculas/minúsculas.
¡Nunca empezar con un
número!

Cierre. Conveniente para
finalizar las sentencias.

La Fábrica de Datos: Tipos y Typecasting

Tipos Primitivos
(No Tipado)



Numéricas,
Arrays,
Objetos,
Undefined,
Null.



toString()

→ Convierte a texto.



parseInt()

→ Filtra hacia número entero.



parseFloat()

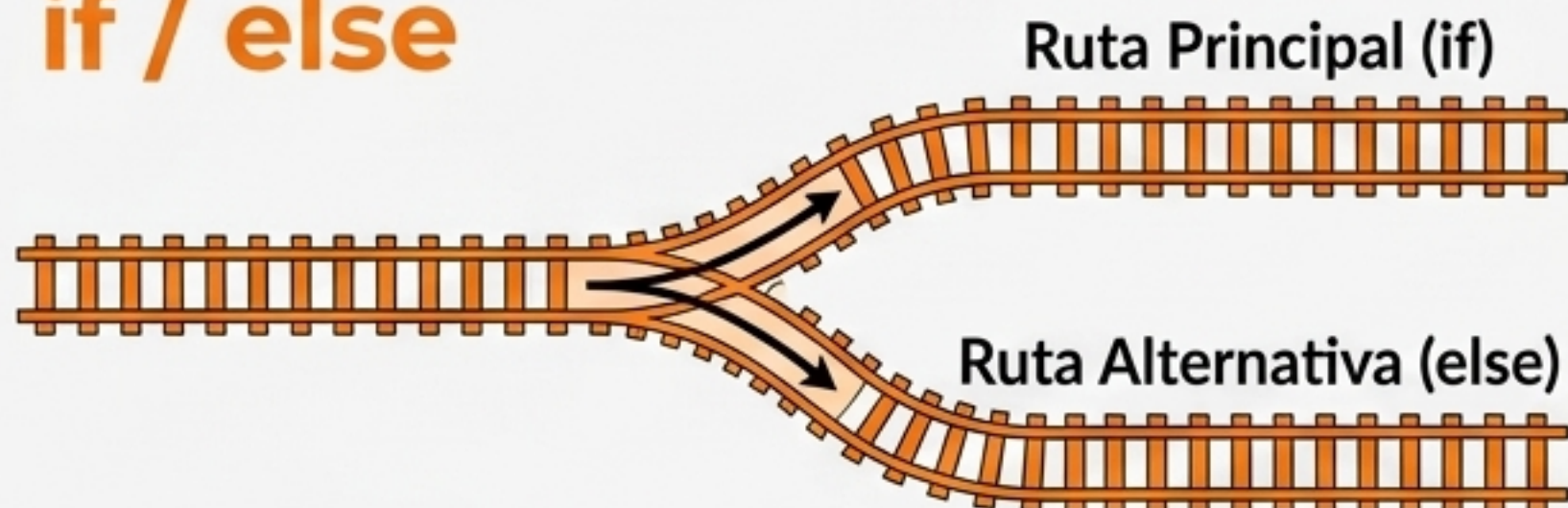
→ Filtra hacia número decimal.



Typecasting
(Conversión)

Estructuras de Control I: Decisiones y Rutas

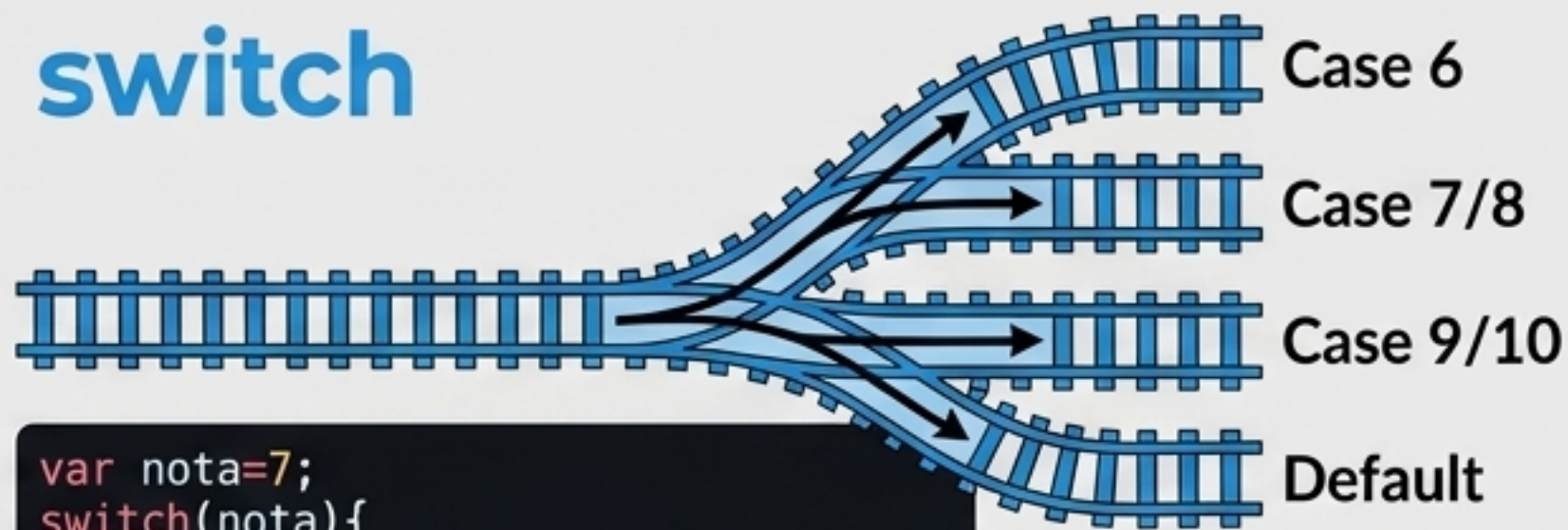
if / else



```
var num1=3;
var num2=4;
if (num1>num2){
    console.log(num1 + " es mayor que " + num2);
}
else{
    console.log(num1 + " es menor que " + num2);
}
```

Bifurcación Binaria: Evalúa una condición. Si es cierta, toma la ruta principal; si no, toma la ruta alternativa (else).

switch



```
var nota=7;
switch(nota){
    case 6:
        console.log("Bien");
        break;
    case 7:
    case 8:
        console.log("Notable");
        break;
    case 9:
    case 10:
        console.log("Sobresaliente");
        break;
    default:
        console.log("Suspenso");
}
```

Decisiones Múltiples: Evalúa una variable y la encarrila por el case correspondiente. Requiere un break para no seguir ejecutando.

Estructuras de Control II: Ciclos y Repetición

El Ciclo Controlado (for)



```
for (inicial; condición; actualización)
```

Sabemos exactamente cuántas vueltas dará.
Ideal para crear tablas.

El Bucle Condicional (while)

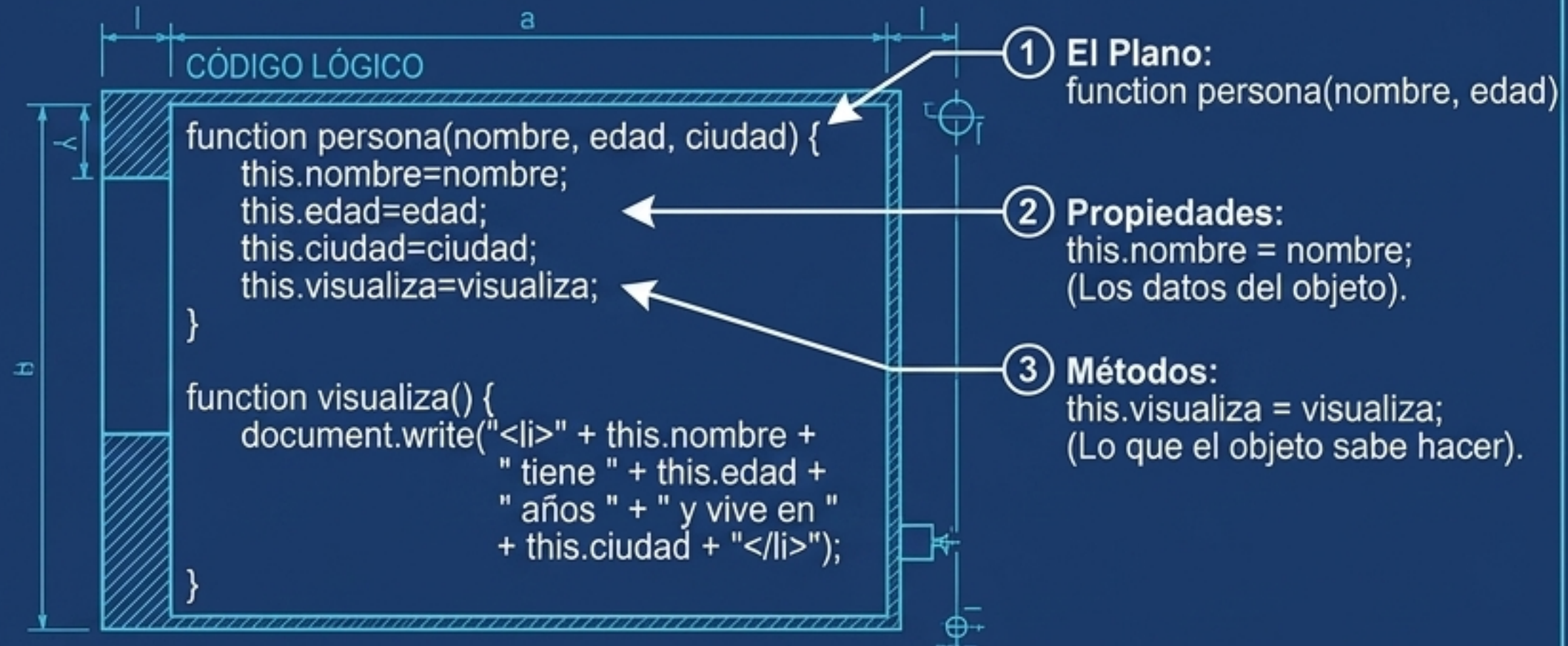
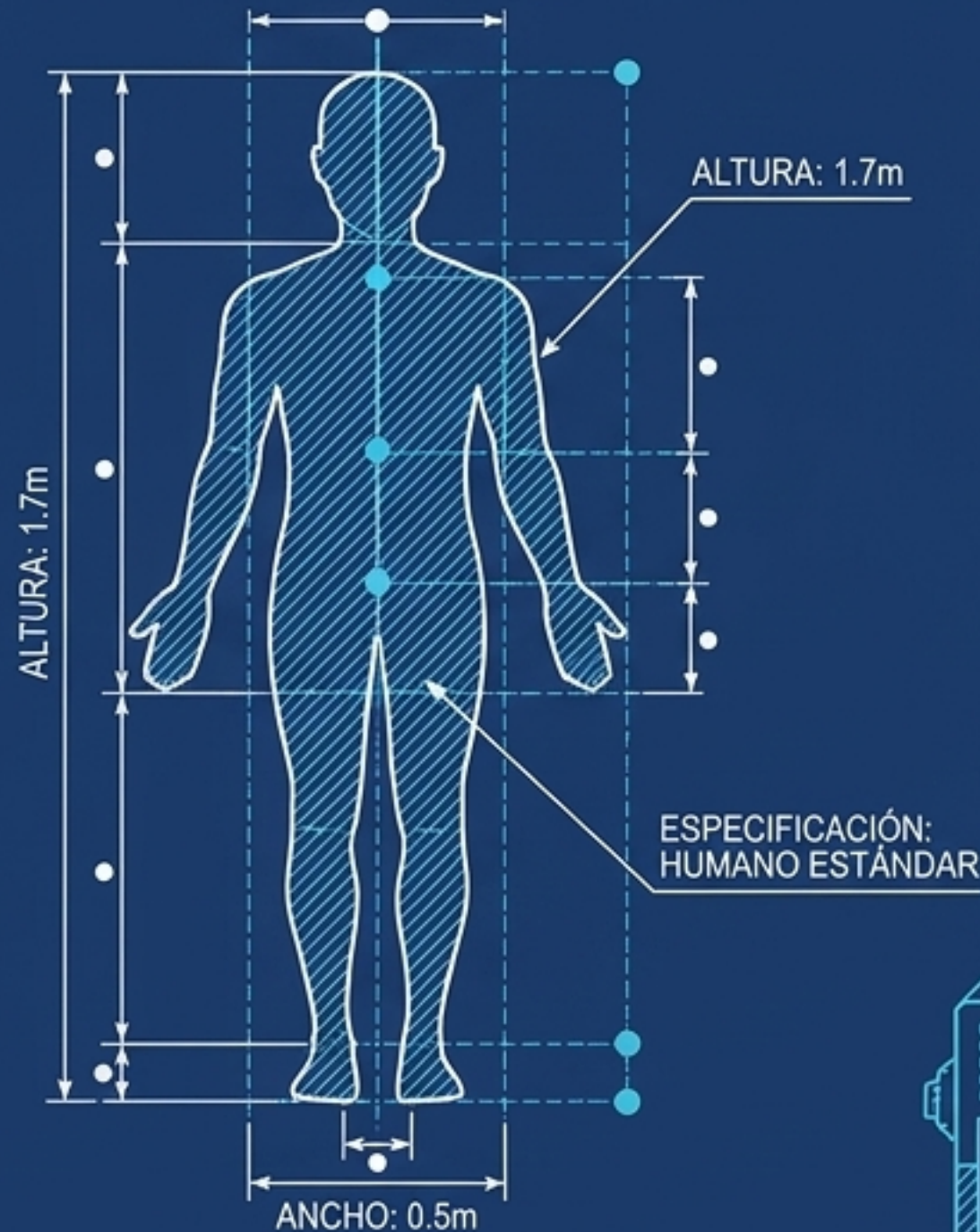


```
while (condición) { ... }
```

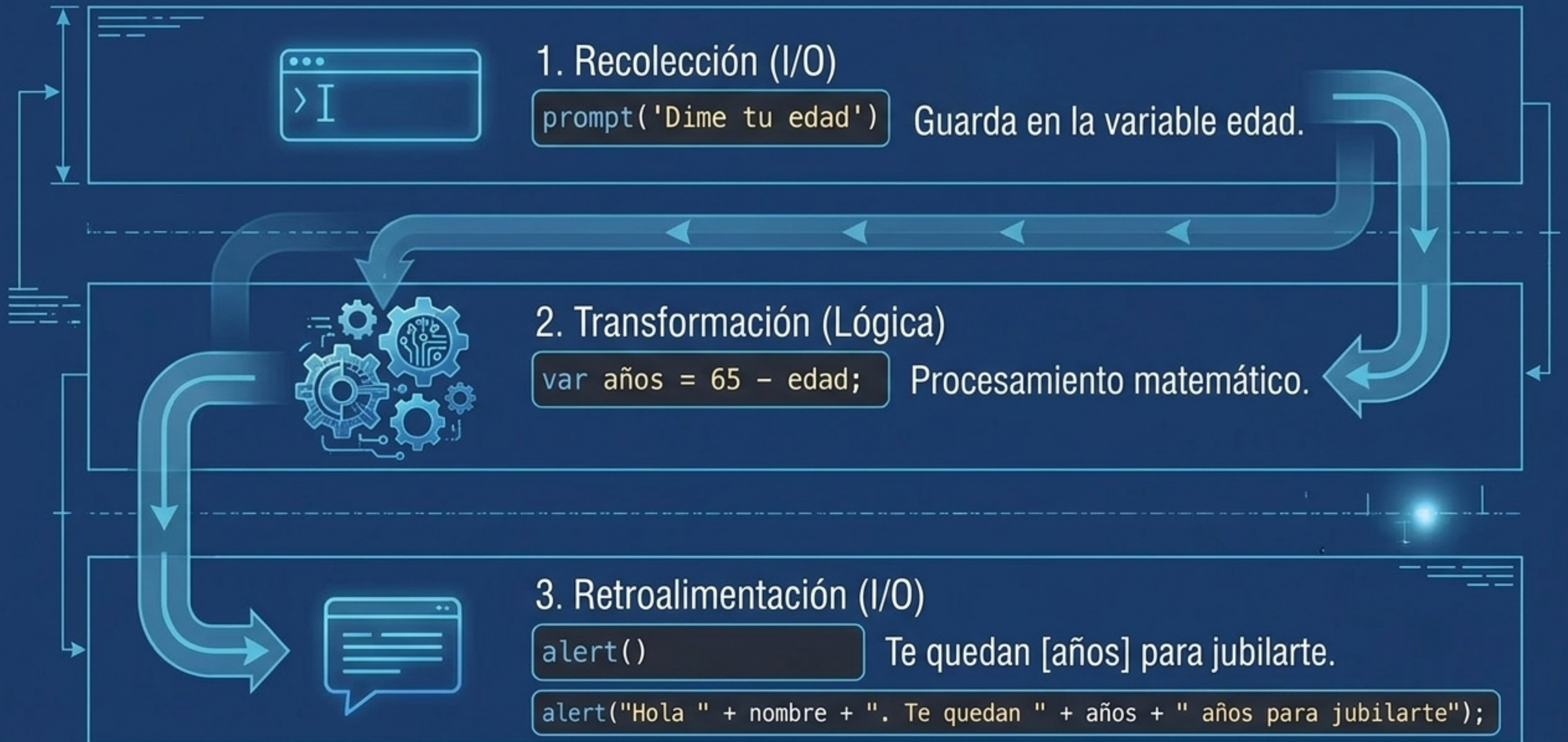
Se repite mientras la condición sea cierta. Requiere actualizar la variable de control dentro del bucle.

Modelando la Realidad: Objetos

Clase: Molde reutilizable



Ensamblaje Práctico 1: El Algoritmo de Jubilación



Ensamblaje Práctico 2: La Letra del DNI



El Plano Final: Arquitectura de JavaScript

